

main

April 2, 2025

0.1 Framework Overview

This framework is designed to streamline the **data preprocessing pipeline** by implementing multiple algorithms for:

- Data Cleansing
- Outlier Detection & Handling
- Feature Selection
- Model Selection

At each step, we evaluate the effectiveness of different algorithms using a **shallow Decision Tree model** and a **KNN Model**. This allows us to determine the best preprocessing strategy based on performance.

0.2 Importing Packages

```
[9]: import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split #Weak Model Test
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score #Weak Model Test
from sklearn.tree import DecisionTreeClassifier #Weak Model Test
from sklearn.neighbors import KNeighborsClassifier #Weak Model Test
from sklearn.impute import SimpleImputer, KNNImputer #Handle Missing
from sklearn.linear_model import LinearRegression #Handle Missing
from sklearn.experimental import enable_iterative_imputer #Handle Missing(Imported Because of API Change)
from sklearn.impute import IterativeImputer #Handle Missing
from sklearn.neighbors import LocalOutlierFactor #Tame Outlier
from sklearn.ensemble import IsolationForest #Tame Outlier
from scipy.stats import zscore #Tame Outlier
```

0.3 Convert into Numeric Values

```
[10]: def ConvertToNumeric(data):
    cols = data.columns
    num_cols = data._get_numeric_data().columns
    categorical_columns = list(set(cols) - set(num_cols))

    for column in categorical_columns:
        categories = list(data[column].astype(str).unique())
        data[column] = data[column].map(lambda x: categories.index(str(x)))

    return data
```

0.4 Loading Data

```
[11]: def load_data(path, y_column):
    data=pd.read_csv(path)
    data=ConvertToNumeric(data)
    Y=data[y_column]
    X=data.drop(columns=y_column)
    return X,Y
```

0.5 Implementing Test Models

```
[12]: def train_and_evaluate(X, y):
    """Trains weak models (KNN and Decision Tree) and evaluates them with
    ↪multiple metrics."""

    print(len(X),len(y),X.isna().sum().sum(),y.isna().sum().sum())

    X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
    ↪random_state=42)
    knn_model = KNeighborsClassifier(n_neighbors=3)
    knn_model.fit(X_train, y_train)
    knn_pred = knn_model.predict(X_test)

    knn_accuracy = accuracy_score(y_test, knn_pred)
    knn_precision = precision_score(y_test, knn_pred, average='weighted',
    ↪zero_division=1)
    knn_recall = recall_score(y_test, knn_pred, average='weighted',
    ↪zero_division=1)
    knn_f1 = f1_score(y_test, knn_pred, average='weighted')

    dt_model = DecisionTreeClassifier(max_depth=3, random_state=42)
    dt_model.fit(X_train, y_train)
    dt_pred = dt_model.predict(X_test)
```

```

    dt_accuracy = accuracy_score(y_test, dt_pred)
    dt_precision = precision_score(y_test, dt_pred, average='weighted',
↪zero_division=1)
    dt_recall = recall_score(y_test, dt_pred, average='weighted',
↪zero_division=1)
    dt_f1 = f1_score(y_test, dt_pred, average='weighted')

    print("KNN (k=3) Metrics:")
    print(f"  Accuracy: {knn_accuracy:.4f}, Precision: {knn_precision:.4f},
↪Recall: {knn_recall:.4f}, F1-score: {knn_f1:.4f}")

    print("\nDecision Tree (max_depth=3) Metrics:")
    print(f"  Accuracy: {dt_accuracy:.4f}, Precision: {dt_precision:.4f},
↪Recall: {dt_recall:.4f}, F1-score: {dt_f1:.4f}")

    weighted_score = (knn_accuracy + knn_precision + knn_recall + knn_f1) + 2 *
↪(dt_accuracy + dt_precision + dt_recall + dt_f1)

    return weighted_score

```

1 Handling Missing Data

These functions help clean our dataset by handling missing values efficiently and selecting the best imputation method.

1.1 Available Algorithms

- **Dropping Methods:**
 - Drop columns with null values exceeding a threshold
 - Drop rows with missing values
- **Statistical Imputation:**
 - Impute with **mean** or **median**
 - Impute with **class-specific mean** or **median**
- **Forward & Backward Filling:**
 - **Forward fill (ffill)**
 - **Backward fill (bfill)**
 - **Interpolate** missing values
- **Model-Based Imputation:**
 - **Model Imputation** (predict missing values using a simple model)
 - **Iterative Model Imputation** (refines predictions iteratively)
 - **KNN Imputation** (fills missing values based on k-nearest neighbors)

```

[13]: def drop_rows(X, Y):
        """Drops rows with any missing values."""
        mask=X.notna().all(axis=1)
        return X[mask], Y[mask]

def impute_mean(X, Y):
        """Fills missing values with column mean while leaving non-missing values
        ↪intact."""

        imputer = SimpleImputer(strategy="mean")
        X_imputed = imputer.fit_transform(X)
        X_imputed_df = pd.DataFrame(X_imputed, columns=X.columns)

        X_filled = X.copy()
        X_filled[X.isnull()] = X_imputed_df[X.isnull()]

        return X_filled, Y

def impute_median(X, Y):
        """Fills missing values with column median while leaving non-missing values
        ↪intact."""

        imputer = SimpleImputer(strategy="median")
        X_imputed = imputer.fit_transform(X)
        X_imputed_df = pd.DataFrame(X_imputed, columns=X.columns)
        X_filled = X.copy()
        X_filled[X.isnull()] = X_imputed_df[X.isnull()]

        return X_filled, Y

def impute_class_mean(X, Y):
        """Fills missing values with the mean of each class in Y, ensuring no NaNs
        ↪remain."""
        X_copy = X.copy()
        for col in X.columns:
            grouped_means = X.groupby(Y)[col].transform(lambda x: x.mean())
            X_copy[col] = X[col].fillna(grouped_means) # Fill NaNs with class mean

            if X_copy[col].isna().sum() > 0:
                X_copy[col].fillna(X[col].mean(), inplace=True)

        return X_copy, Y

def impute_class_median(X, Y):
        """Fills missing values with the median of each class in Y."""
        X_copy = X.copy()
        for col in X.columns:

```

```

        X_copy[col] = X.groupby(Y)[col].transform(lambda x: x.fillna(x.
↪median()))

        if X_copy[col].isna().sum() > 0:
            X_copy[col].fillna(X[col].mean(), inplace=True)
    return X_copy, Y

def ffill(X, Y):
    """Fills missing values with the previous row (forward fill) and ensures no_
↪NaNs remain."""
    X_filled = X.ffill()
    X_filled = X_filled.bfill()

    return X_filled, Y

def bfill(X, Y):
    """Fills missing values with the next row (backward fill)."""
    X_filled = X.bfill()
    X_filled = X_filled.ffill()
    return X_filled, Y

def interpolate(X, Y):
    """Interpolates missing values linearly and ensures no NaNs remain."""
    X_filled = X.interpolate(method="linear", limit_direction="both")
    return X_filled, Y

def Model_imputation(X, Y):
    """Uses a simple regression model to impute missing values."""

    X_copy = X.copy()

    for col in X.columns:
        missing_mask = X_copy[col].isna()
        if missing_mask.sum() > 0:
            known_data = X_copy.dropna()
            known_X = known_data.drop(columns=[col])
            known_y = known_data[col]
            missing_X = X_copy.loc[missing_mask].drop(columns=[col])

            if missing_X.isna().sum().sum() > 0:
                missing_X = missing_X.fillna(known_X.mean())

            if len(known_X) > 0 and len(missing_X) > 0:
                model = LinearRegression()
                model.fit(known_X, known_y)

```

```

        X_copy.loc[missing_mask, col] = model.predict(missing_X)

    return X_copy, Y

def Iterative_model_Imputation(X, Y):
    """Uses iterative imputation (sklearn's IterativeImputer) to fill only null_
    ↪ values."""

    imputer = IterativeImputer()
    X_imputed = imputer.fit_transform(X)
    X_imputed_df = pd.DataFrame(X_imputed, columns=X.columns)

    X_filled = X.copy()
    X_filled[X.isnull()] = X_imputed_df[X.isnull()]

    return X_filled, Y

def KNN_Imputation(X, Y, n_neighbors=5):
    """Uses KNN to fill missing values while leaving non-missing values intact.
    ↪ """

    imputer = KNNImputer(n_neighbors=n_neighbors)
    X_imputed = imputer.fit_transform(X)
    X_imputed_df = pd.DataFrame(X_imputed, columns=X.columns)

    X_filled = X.copy()
    X_filled[X.isnull()] = X_imputed_df[X.isnull()]

    return X_filled, Y

algorithm_functions_clean_data = {
    "drop_rows": drop_rows,
    "impute_mean": impute_mean,
    "impute_median": impute_median,
    "impute_class_mean": impute_class_mean,
    "impute_class_median": impute_class_median,
    "ffill": ffill,
    "bfill": bfill,
    "interpolate": interpolate,
    "Model_imputation": Model_imputation,
    "Iterative_model_Imputation": Iterative_model_Imputation,
    "KNN_Imputation": KNN_Imputation,
}

```

```

[14]: def drop_columns(X, threshold=0.3):
    return X.dropna(axis=1, thresh=(len(X)*threshold), inplace=False)

```

```

def handling_missing_data(X, Y):
    """Applies different missing data handling algorithms, selects the best_
    ↪ one, and returns the transformed dataset."""
    X = drop_columns(X)
    Y = Y.dropna(inplace=False)
    X = X.loc[Y.index]
    x_copy = X.copy()
    y_copy = Y.copy()
    acc_holder = {}

    best_algo = None
    best_accuracy = -1

    for name, func in algorithm_functions_clean_data.items():
        X_transformed, Y_transformed = func(x_copy.copy(), y_copy.copy())
        print(""*10, name, ""*10)

        accuracy = train_and_evaluate(X_transformed, Y_transformed)
        acc_holder[name] = accuracy
        print(""*10, accuracy, ""*10)

        if accuracy > best_accuracy:
            best_accuracy = accuracy
            best_algo = name
    if best_algo:
        x_copy, y_copy = algorithm_functions_clean_data[best_algo](x_copy.
    ↪ copy(), y_copy.copy())

    return x_copy, y_copy, best_algo, best_accuracy, acc_holder

```

2 Taming Outliers

These functions help clean our dataset from outliers efficiently and selecting the best outlier detection algorithm.

2.1 Available Algorithms

- **IQR Method(1.5):**
 - Take 1.5 times the IQR and then subtract this value from Q1 and add this value to Q3
- **LOF:**
 - For any data object **q**, the **LOF score** is computed as the ratio of the **average local density** of its **k-nearest neighbors** to its **own local density** [25].

$$LOF(q) = \frac{\sum_{x \in N_k(q)} lrd(x)}{|N_k(q)| \times lrd(q)}$$

where the **local reachability density (lrd)** of $\mathbf{q}$ is given by:

$$lrd(q) = \frac{|N_k(q)|}{\sum_{x \in N_k(q)} \max(\text{dist}_k(x, D), \text{dist}(q, x))}$$

- **SP:**

- Employ a **scoring measure** based on the nearest neighbor ($\mathbf{k} = \mathbf{1}$) within random sub-samples ($\mathbf{S} \quad \mathbf{D}$).

$$S_p(q) = \min_{x \in S} \text{dist}(q, x)$$

where $\text{dist}(\mathbf{q}, \mathbf{x})$ represents the distance between $\mathbf{q}$ and $\mathbf{x}$.

- **iForest:**

- A **random split** is performed on a randomly selected feature.
- The partitioning continues until either:
 - * Each node contains only **one data object**, or
 - * The tree reaches its **height limit**.

$$iForest(q) = \frac{1}{t} \sum_{i=1}^t l_i(q)$$

- **iNNe:**

- This method builds **hyperspheres** using all dimensions of the dataset. The **isolation score** of a data object $\mathbf{q}$ is defined as:

$$I(q) = \begin{cases} \tau(\eta_{cnn}(q)), & \text{if } q \in \bigcup_{c \in S} B(c) \\ 1 - \tau(cnn(q)), & \text{otherwise} \end{cases}$$

```
[ ]: def IQR(X, Y):
    """Adjust outliers based on IQR min-max whiskers."""
    Q1 = X.quantile(0.25)
    Q3 = X.quantile(0.75)
    IQR = Q3 - Q1
    min_whisker = Q1 - 1.5 * IQR
    max_whisker = Q3 + 1.5 * IQR

    X_adjusted = pd.DataFrame(X.clip(lower=min_whisker, upper=max_whisker,
↪axis=1))
    return X_adjusted, Y
```

```

def LOF(X, Y):
    """Adjust based on LOF anomaly score."""
    clf = LocalOutlierFactor(n_neighbors=20, contamination=0.1)
    anomaly_scores = clf.fit_predict(X)
    X_scores = clf.negative_outlier_factor_

    adjustment_factor = np.abs(X_scores) / np.max(np.abs(X_scores))

    X_adjusted = X * (1 - adjustment_factor[:, np.newaxis])
    return X_adjusted, Y

def IsolationForestOutlier(X, Y):
    """Adjust outliers based on Isolation Forest isolation score."""
    iso = IsolationForest(n_estimators=100, contamination=0.1, random_state=42)
    iso.fit(X)

    scores = iso.decision_function(X)

    adjustment_factor = (scores - scores.min()) / (scores.max() - scores.min())
    X_adjusted = X * (1 - adjustment_factor[:, np.newaxis])

    return X_adjusted, Y

def SP(X, Y):
    """Adjusts values based on Standardization Projection (capping beyond 3 std_
    ↪ dev)."""

    Z_scores = np.abs(zscore(X, nan_policy='omit'))

    cap_value = 3
    Z_scores = np.clip(Z_scores, None, cap_value)

    Z_scores = np.nan_to_num(Z_scores, nan=0)

    max_Z = np.max(Z_scores, axis=0)
    max_Z[max_Z == 0] = 1

    X_adjusted = X.copy()

    for col in X.columns:
        mask = ~X_adjusted[col].isna()
        X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask, X.
    ↪ columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])

    return X_adjusted, Y

```

```

def IsolationNNe(X, Y):
    """Adjust outliers based on Isolation Forest distance score."""

    iso = IsolationForest(contamination=0.05, n_estimators=200, random_state=42)
    iso.fit(X)

    dist = iso.decision_function(X)

    adjustment_factor = np.abs(dist) / np.max(np.abs(dist))
    X_adjusted = X * (1 - adjustment_factor[:, np.newaxis])

    return X_adjusted, Y

algorithm_functions_tame_outlier = {
    "IQR": IQR,
    "LOF": LOF,
    "iForest": IsolationForestOutlier,
    "SP": SP,
    "iNNe": IsolationNNe,
}

```

```

[16]: def taming_outliers(X,Y):
    acc_holder = {}
    x_copy = X.copy()
    y_copy = Y.copy()
    best_algo = None
    best_accuracy = 0

    for name, func in algorithm_functions_tame_outlier.items():
        X_Adjusted,Y_Adjusted = func(x_copy.copy(), y_copy.copy())
        print(name)
        accuracy = train_and_evaluate(X_Adjusted, Y_Adjusted)
        acc_holder[name] = accuracy
        print(accuracy)

        if accuracy > best_accuracy:
            best_accuracy = accuracy
            best_algo = name
    if best_algo:
        x_copy,y_copy = algorithm_functions_tame_outlier[best_algo](x_copy.
↪copy(), y_copy.copy())

    return x_copy, y_copy, best_algo, best_accuracy,acc_holder

```

```
[17]: X,Y=load_data(path='data1_1.csv',y_column='smokingstatus')

[18]: x_copy_clean_data, y_copy_clean_data, best_algo_clean_data, \
      ↪best_accuracy_clean_data,acc_holder_clean_data=handling_missing_data(X,Y)
```

```
***** drop_rows *****
100 100 0 0
KNN (k=3) Metrics:
  Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
  Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
***** 9.086904761904762 *****
***** impute_mean *****
100 100 0 0
KNN (k=3) Metrics:
  Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
  Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
***** 9.086904761904762 *****
***** impute_median *****
100 100 0 0
KNN (k=3) Metrics:
  Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
  Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
***** 9.086904761904762 *****
***** impute_class_mean *****
100 100 0 0
KNN (k=3) Metrics:
  Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
  Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
***** 9.086904761904762 *****
***** impute_class_median *****
100 100 0 0
KNN (k=3) Metrics:
  Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
  Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
***** 9.086904761904762 *****
***** ffill *****
100 100 0 0
KNN (k=3) Metrics:
```

```

    Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
    Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
    ***** 9.086904761904762 *****
    ***** bfill *****
    100 100 0 0
KNN (k=3) Metrics:
    Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
    Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
    ***** 9.086904761904762 *****
    ***** interpolate *****
    100 100 0 0
KNN (k=3) Metrics:
    Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
    Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
    ***** 9.086904761904762 *****
    ***** Model_imputation *****
    100 100 0 0
KNN (k=3) Metrics:
    Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
    Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
    ***** 9.086904761904762 *****
    ***** Iterative_model_Imputation *****
    100 100 0 0
KNN (k=3) Metrics:
    Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
    Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
    ***** 9.086904761904762 *****
    ***** KNN_Imputation *****
    100 100 0 0
KNN (k=3) Metrics:
    Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:
    Accuracy: 0.7500, Precision: 0.7500, Recall: 0.7500, F1-score: 0.7500
    ***** 9.086904761904762 *****

```

```
[19]: x_copy_tame_outlier, y_copy_tame_outlier, best_algo_tame_outlier,
      ↪best_accuracy_tame_outlier, acc_holder_tame_outlier=taming_outliers(x_copy_clean_data, y_copy
```

IQR

100 100 0 0

KNN (k=3) Metrics:

Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:

Accuracy: 0.8000, Precision: 0.7556, Recall: 0.8000, F1-score: 0.7771

9.352301587301588

LOF

100 100 0 0

KNN (k=3) Metrics:

Accuracy: 0.7500, Precision: 0.8583, Recall: 0.7500, F1-score: 0.7286

Decision Tree (max_depth=3) Metrics:

Accuracy: 0.7000, Precision: 0.8000, Recall: 0.7000, F1-score: 0.7000

8.886904761904763

iForest

100 100 0 0

KNN (k=3) Metrics:

Accuracy: 0.4000, Precision: 0.6800, Recall: 0.4000, F1-score: 0.5037

Decision Tree (max_depth=3) Metrics:

Accuracy: 0.6500, Precision: 0.6906, Recall: 0.6500, F1-score: 0.6697

7.304347643097643

SP

100 100 0 0

KNN (k=3) Metrics:

Accuracy: 0.8500, Precision: 0.8725, Recall: 0.8500, F1-score: 0.7811

Decision Tree (max_depth=3) Metrics:

Accuracy: 0.8000, Precision: 0.8000, Recall: 0.8000, F1-score: 0.8000

9.75358108108108

/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[' 1.          1.95959596   2.87878788   3.75757576   4.5959596
 5.39393939   6.15151515   6.86868687   7.54545455   8.18181818
 8.77777778   9.33333333   9.84848485  10.32323232  10.75757576
11.15151515  11.50505051  11.81818182  12.09090909  12.32323232
12.51515152  12.66666667  12.77777778  12.84848485  12.87878788
12.86868687  12.81818182  12.72727273  12.5959596   12.42424242
12.21212121  11.95959596  11.66666667  11.33333333  10.95959596
10.54545455  10.09090909   9.5959596   9.06060606   8.48484848
 7.86868687   7.21212121   6.51515152   5.77777778   5.
 4.18181818   3.32323232   2.42424242   1.48484848   0.50505051
 0.51515152   1.57575758   2.67676768   3.81818182   5.
 6.22222222   7.48484848   8.78787879  10.13131313  11.51515152
12.93939394  14.4040404   15.90909091  17.45454545  19.04040404
20.66666667  22.33333333  24.04040404  25.78787879  27.57575758
29.4040404   31.27272727  33.18181818  35.13131313  37.12121212
39.15151515  41.22222222  43.33333333  45.48484848  47.67676768
49.90909091  52.18181818  54.49494949  56.84848485  59.24242424
61.67676768  64.15151515  66.66666667  69.22222222  71.81818182
74.45454545  77.13131313  79.84848485  82.60606061  85.4040404
88.24242424  91.12121212  94.04040404  97.          100.          ]' has dtype
incompatible with int64, please explicitly cast to a compatible dtype first.
```

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame,

like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values`` instead, to perform the assignment in a single step and ensure this keeps updating the original `df``.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104
```

```
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 2. 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
2. 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 2.
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104]' has dtype incompatible with
int64, please explicitly cast to a compatible dtype first.
```

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will

behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[0.61290323 0.61290323 0.        0.61290323 0.61290323
0.61290323
0.        0.        0.61290323 0.61290323 0.        0.
0.        0.        0.61290323 0.61290323 0.        0.
0.61290323 0.61290323 0.        0.61290323 0.        0.61290323
0.        0.61290323 0.61290323 0.        0.61290323 0.
0.        0.61290323 0.61290323 0.        0.61290323 0.
0.        0.61290323 0.61290323 0.        0.61290323 0.61290323
0.61290323 0.61290323 0.61290323 0.61290323 0.61290323 0.61290323
0.61290323 0.61290323 0.61290323 0.        0.        0.61290323
0.61290323 0.61290323 0.        0.        ]' has dtype incompatible with
int64, please explicitly cast to a compatible dtype first.
```

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default

behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[ 8.75440658 13.01645123 32.26321974  1.49353702  0.
19.35722679
```

```
1.49941246 17.70152761 1.49941246 11.54876616 7.40658049 16.09283196
0.54289072 1.49941246 3.72972973 0.54289072 3.25381904 10.12808461
14.53113984 28.34077556 0. 8.75440658 0.54289072 4.06345476
1.49941246 17.70152761 0.54289072 24.60634548 0. 34.29494712
4.91539365 0. 0. 30.27849589 26.45005875 0.
0. 0. 0. 14.53113984 3.25381904 3.72972973
16.09283196 8.75440658 10.12808461 0.45475911 0.54289072 10.12808461
19.35722679 6.1480611 1.49353702 2.59106933 0.54289072 3.25381904
2.59106933 3.72972973 14.53113984 4.91539365 8.75440658 30.27849589
2.59106933 0.54289072 10.12808461 32.26321974 17.70152761 0.45475911
22.80963572 1.49941246 3.25381904 2.59106933 30.27849589 4.82608696
1.49941246 17.70152761 7.42773208 24.60634548 11.54876616 3.72972973
8.75440658 26.45005875 10.12808461 0. 3.72972973 13.01645123
8.75440658 4.91539365 0. 4.06345476 2.59106933 24.60634548
0. 16.09283196 0. 6.21034078 1.49353702 3.72972973
10.12808461 3.25381904 2.39717979 4.91539365]' has dtype incompatible with
int64, please explicitly cast to a compatible dtype first.
```

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[1.53093954e+01 1.60248511e+01 1.12506254e+01 1.07148814e+01
0.00000000e+00 6.28651584e+00 1.87171346e+01 5.13025808e+00
5.36083147e+00 9.86718506e-01 3.70273749e+00 1.82763325e+00
5.36083147e+00 9.86718506e-01 1.62757692e-01 6.28651584e+00
1.53806019e+01 1.43023322e+01 2.22062526e+01 1.62757692e-01
0.00000000e+00 1.64758255e+01 2.63000000e+02 1.64758255e+01
1.12506254e+01 7.22915414e+00 3.09409154e+01 2.68550191e+00
2.58479559e+01 1.11692466e+01 1.43430216e+00 0.00000000e+00
6.44249197e-01 2.58479559e+01 2.68550191e+00 0.00000000e+00
2.68550191e+00 5.36083147e+00 1.43430216e+00 3.35891186e+01
1.43023322e+01 5.36083147e+00 3.70273749e+00 2.70957649e+01
5.13025808e+00 4.42497474e+00 6.28651584e+00 1.21834305e+02
9.16529252e+00 1.01587926e+01 6.28651584e+00 7.14438451e+00
1.43430216e+00 1.43430216e+00 1.32410164e+01 1.17694156e+01
3.70273749e+00 2.96354630e+00 6.44249197e-01 8.40236584e+00
6.44249197e-01 4.78473706e+01 4.63452527e+01 6.44249197e-01
5.13025808e+00 1.01621834e+01 1.35271987e+02 1.43430216e+00
8.40236584e+00 0.00000000e+00 1.64758255e+01 2.68550191e+00
6.44249197e-01 7.41869904e+01 2.20740119e+00 1.62757692e-01
3.70273749e+00 5.81858748e+00 9.86718506e-01 9.67899648e+01
1.87171346e+01 0.00000000e+00 2.96354630e+00 3.63051376e+01
```

```
1.62757692e-01 5.13025808e+00 0.00000000e+00 9.16529252e+00
6.48996296e+00 2.68550191e+00 0.00000000e+00 3.09409154e+01
0.00000000e+00 9.86718506e-01 2.96354630e+00 6.28651584e+00
9.87837465e+01 2.96354630e+00 7.22915414e+00 1.62757692e-01]' has dtype
incompatible with int64, please explicitly cast to a compatible dtype first.
```

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values`` instead, to perform the assignment in a single step and ensure this keeps updating the original `df``.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[4.47834456e+00 6.73339750e+00 9.20115496e+00 1.10529355e+01
0.00000000e+00 3.22906641e+00 1.63282002e+01 2.78248316e+00
1.78498556e+01 2.26458133e+01 1.78498556e+01 5.26852743e+00
2.78248316e+00 3.22906641e+00 5.00481232e-02 9.29740135e+00
9.29740135e+00 3.51838306e+01 2.43214629e+01 4.36573628e+00
0.00000000e+00 1.19942252e+01 8.64196343e+00 1.06265640e+01
1.02040423e+01 5.54090472e+00 2.43214629e+01 7.40808470e+00
1.78498556e+01 5.54090472e+00 9.32627526e-01 0.00000000e+00
9.29740135e+00 2.26458133e+01 3.91126083e+01 0.00000000e+00
2.26458133e+01 2.13089509e+00 2.60356112e+01 2.60356112e+01
1.63282002e+01 2.26458133e+01 5.54090472e+00 5.86977863e+01
1.63282002e+01 5.54090472e+00 1.06265640e+01 1.94100096e+01
```

```

5.00481232e-02 2.10086622e+01 1.94100096e+01 5.26852743e+00
1.63282002e+01 1.87680462e+00 5.54090472e+00 2.78248316e+00
1.07122233e+00 4.47834456e+00 2.26458133e+01 8.04427334e+00
9.29740135e+00 1.06265640e+01 1.07122233e+00 9.29740135e+00
6.02021174e+00 6.73339750e+00 5.54090472e+00 9.20115496e+00
1.78498556e+01 0.00000000e+00 1.63282002e+01 6.75457170e+00
1.06265640e+01 1.14196343e+01 8.04427334e+00 1.20375361e+01
5.00481232e-02 4.36573628e+00 1.19942252e+01 8.00673725e+00
6.75457170e+00 0.00000000e+00 9.32627526e-01 1.19942252e+01
4.36573628e+00 6.75457170e+00 0.00000000e+00 1.87680462e+00
4.36573628e+00 1.07122233e+00 0.00000000e+00 1.63282002e+01
0.00000000e+00 9.29740135e+00 1.87680462e+00 1.48450433e+01
9.32627526e-01 1.63282002e+01 3.22906641e+00 9.29740135e+00]' has dtype
incompatible with int64, please explicitly cast to a compatible dtype first.

```

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[1.53770156e+01 1.51206096e+01 1.47398135e+01 3.35118379e+01
0.00000000e+00 1.29444471e+01 1.79625401e+00 2.03238152e+01
1.53301726e+01 1.04534156e+01 9.25446043e+00 1.11296334e+01

```

```

4.87656334e+01 4.18379110e+01 4.77517505e+01 1.36330111e+01
6.74377987e+00 1.53770156e+01 2.75332816e+00 1.51693086e+01
0.00000000e+00 3.29918442e+00 4.89000000e+02 8.16784376e+01
1.01657809e+01 5.08176075e+01 1.44574603e+02 7.91907145e+01
2.18593461e+01 3.62145708e+01 4.04136978e+00 0.00000000e+00
3.16753175e+00 3.29918442e+00 1.54340261e+01 0.00000000e+00
1.54040885e+01 1.20097656e+01 1.32302258e+00 3.52351443e-01
2.58393883e+01 1.54040885e+01 2.03876242e+02 8.04305413e+01
2.83242467e+01 1.30848147e+01 2.91686716e+01 1.15498406e+02
2.07655073e+02 1.76563861e+02 1.25271771e+01 1.45115294e+01
1.50217188e+01 4.98516973e+00 1.50451202e+01 2.58393883e+01
1.41085827e+01 1.17705475e+01 1.52309994e+01 1.30848147e+01
1.45088262e-01 4.04136978e+00 2.71850859e+00 1.54230920e+01
1.53418733e+01 1.50995484e+01 1.68582393e+00 5.69221275e+00
1.53770156e+01 0.00000000e+00 1.30848147e+01 3.99311866e+01
8.41721723e-01 3.28920496e+02 1.68582393e+00 3.44046795e+01
5.71671960e+01 4.46618466e+00 6.50597392e-01 9.19521067e+01
7.57420440e+00 0.00000000e+00 2.18593461e+01 2.72506137e+02
1.49615613e+01 1.47398135e+01 0.00000000e+00 1.39581283e+01
1.07329808e+01 5.71671960e+01 0.00000000e+00 1.52036440e+01
0.00000000e+00 4.88293012e+00 1.28859438e+01 8.41721723e-01
8.54705428e+01 1.41085827e+01 4.88293012e+00 2.50272410e+01]' has dtype
incompatible with int64, please explicitly cast to a compatible dtype first.

```

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[2.87247247e+01 3.43163163e+01 2.17757758e+01 1.99319319e+01
0.00000000e+00 3.58638639e+01 6.82722723e+01 1.93793794e+00
1.74614615e+01 1.53413413e+01 7.16716717e-01 8.23623624e+00
1.96216216e+01 1.74614615e+01 7.26126126e+00 2.98378378e+01
4.47207207e+01 4.47207207e+01 7.57497497e+01 3.46146146e+00
0.00000000e+00 6.10450450e+01 3.78000000e+02 6.24704705e+01
2.57277277e+01 2.07167167e+01 1.18666667e+02 9.22122122e+00
9.63623624e+01 3.46386386e+01 1.78178178e-01 0.00000000e+00
2.80280280e+00 6.68068068e+01 6.29629630e+00 0.00000000e+00
1.78178178e-01 2.29369369e+01 2.80280280e+00 7.72752753e+01
5.13473473e+01 1.53413413e+01 1.06306306e+00 1.87125125e+02
1.89261261e+02 6.16216216e+00 2.07167167e+01 2.34086086e+02
1.74614615e+01 3.22182182e+01 1.02162162e+01 2.62522523e+01
8.57657658e+00 1.52792793e+01 1.53413413e+01 2.40940941e+01
8.57657658e+00 5.33733734e+00 5.33733734e+00 2.05565566e+01
```

```

9.22122122e+00 8.98398398e+01 5.27027027e+01 3.65765766e+00
1.79979980e+01 2.77657658e+01 1.42662663e+02 1.01361361e+01
2.11711712e+01 0.00000000e+00 1.32612613e+01 1.93793794e+00
8.57657658e+00 6.10450450e+01 1.38598599e+01 1.16556557e+01
2.11711712e+01 1.52792793e+01 1.93793794e+00 1.48440440e+02
1.96216216e+01 0.00000000e+00 1.24004004e+01 2.17941942e+02
1.66586587e+01 1.86526527e+01 0.00000000e+00 1.96216216e+01
2.05565566e+01 1.93793794e+00 0.00000000e+00 5.13473473e+01
0.00000000e+00 3.65765766e+00 1.52792793e+01 1.02162162e+01
7.42342342e+01 6.16216216e+00 2.53653654e+00 8.57657658e+00]' has dtype
incompatible with int64, please explicitly cast to a compatible dtype first.

```

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in
certain cases, but when using Copy-on-Write (which will become the default
behaviour in pandas 3.0) this will never work to update the original DataFrame
or Series, because the intermediate object on which we are setting values will
behave as a copy.
A typical example is when you are setting values in a column of a DataFrame,
like:
```

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])

iNNe
100 100 0 0
KNN (k=3) Metrics:
Accuracy: 0.6000, Precision: 0.6800, Recall: 0.6000, F1-score: 0.6375
```

```
Decision Tree (max_depth=3) Metrics:
Accuracy: 0.6000, Precision: 0.7286, Recall: 0.6000, F1-score: 0.6581
7.690771889400921
```

```
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in
certain cases, but when using Copy-on-Write (which will become the default
behaviour in pandas 3.0) this will never work to update the original DataFrame
or Series, because the intermediate object on which we are setting values will
behave as a copy.
A typical example is when you are setting values in a column of a DataFrame,
like:
```

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[ 1.          1.95959596  2.87878788  3.75757576  4.59595956
 5.39393939  6.15151515  6.86868687  7.54545455  8.18181818
 8.77777778  9.33333333  9.84848485 10.32323232 10.75757576
11.15151515 11.50505051 11.81818182 12.09090909 12.32323232
12.51515152 12.66666667 12.77777778 12.84848485 12.87878788
12.86868687 12.81818182 12.72727273 12.59595956 12.42424242
12.21212121 11.95959596 11.66666667 11.33333333 10.95959596
10.54545455 10.09090909  9.59595956  9.06060606  8.48484848
 7.86868687  7.21212121  6.51515152  5.77777778  5.
 4.18181818  3.32323232  2.42424242  1.48484848  0.50505051
 0.51515152  1.57575758  2.67676768  3.81818182  5.
 6.22222222  7.48484848  8.78787879 10.13131313 11.51515152
12.93939394 14.4040404  15.90909091 17.45454545 19.04040404
20.66666667 22.33333333 24.04040404 25.78787879 27.57575758
29.4040404  31.27272727 33.18181818 35.13131313 37.12121212
39.15151515 41.22222222 43.33333333 45.48484848 47.67676768
49.90909091 52.18181818 54.49494949 56.84848485 59.24242424
61.67676768 64.15151515 66.66666667 69.22222222 71.81818182
74.45454545 77.13131313 79.84848485 82.60606061 85.4040404
88.24242424 91.12121212 94.04040404 97.          100.          ]' has dtype
```

incompatible with int64, please explicitly cast to a compatible dtype first.

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values`` instead, to perform the assignment in a single step and ensure this keeps updating the original `df``.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104
```

```
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 2.          0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
2.          0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 2.
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104 0.05862104 0.05862104
0.05862104 0.05862104 0.05862104 0.05862104]' has dtype incompatible with
int64, please explicitly cast to a compatible dtype first.
```

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame,

like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[0.61290323 0.61290323 0.          0.61290323 0.61290323
0.61290323
```

```
0.          0.          0.61290323 0.61290323 0.          0.
0.          0.61290323 0.61290323 0.          0.61290323 0.61290323
0.61290323 0.61290323 0.61290323 0.          0.61290323 0.61290323
0.          0.          0.61290323 0.          0.61290323 0.
0.61290323 0.61290323 0.          0.61290323 0.          0.61290323
0.          0.61290323 0.61290323 0.          0.61290323 0.
0.          0.61290323 0.          0.          0.61290323 0.61290323
0.          0.61290323 0.          0.          0.61290323 0.61290323
0.61290323 0.61290323 0.61290323 0.          0.61290323 0.
0.61290323 0.          0.61290323 0.          0.61290323 0.61290323
0.61290323 0.          0.61290323 0.61290323 0.61290323 0.
0.61290323 0.61290323 0.61290323 0.          0.          0.61290323
0.61290323 0.61290323 0.          0.          ]' has dtype incompatible with
int64, please explicitly cast to a compatible dtype first.
```

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will

behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[ 8.75440658 13.01645123 32.26321974  1.49353702  0.
19.35722679
```

```
1.49941246 17.70152761 1.49941246 11.54876616 7.40658049 16.09283196
0.54289072 1.49941246 3.72972973 0.54289072 3.25381904 10.12808461
14.53113984 28.34077556 0. 8.75440658 0.54289072 4.06345476
1.49941246 17.70152761 0.54289072 24.60634548 0. 34.29494712
4.91539365 0. 0. 30.27849589 26.45005875 0.
0. 0. 0. 14.53113984 3.25381904 3.72972973
16.09283196 8.75440658 10.12808461 0.45475911 0.54289072 10.12808461
19.35722679 6.1480611 1.49353702 2.59106933 0.54289072 3.25381904
2.59106933 3.72972973 14.53113984 4.91539365 8.75440658 30.27849589
2.59106933 0.54289072 10.12808461 32.26321974 17.70152761 0.45475911
22.80963572 1.49941246 3.25381904 2.59106933 30.27849589 4.82608696
1.49941246 17.70152761 7.42773208 24.60634548 11.54876616 3.72972973
8.75440658 26.45005875 10.12808461 0. 3.72972973 13.01645123
8.75440658 4.91539365 0. 4.06345476 2.59106933 24.60634548
0. 16.09283196 0. 6.21034078 1.49353702 3.72972973
10.12808461 3.25381904 2.39717979 4.91539365]' has dtype incompatible with
int64, please explicitly cast to a compatible dtype first.
```

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default

behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[1.53093954e+01 1.60248511e+01 1.12506254e+01 1.07148814e+01
0.00000000e+00 6.28651584e+00 1.87171346e+01 5.13025808e+00
5.36083147e+00 9.86718506e-01 3.70273749e+00 1.82763325e+00
5.36083147e+00 9.86718506e-01 1.62757692e-01 6.28651584e+00
1.53806019e+01 1.43023322e+01 2.22062526e+01 1.62757692e-01
0.00000000e+00 1.64758255e+01 2.63000000e+02 1.64758255e+01
1.12506254e+01 7.22915414e+00 3.09409154e+01 2.68550191e+00
2.58479559e+01 1.11692466e+01 1.43430216e+00 0.00000000e+00
6.44249197e-01 2.58479559e+01 2.68550191e+00 0.00000000e+00
2.68550191e+00 5.36083147e+00 1.43430216e+00 3.35891186e+01
1.43023322e+01 5.36083147e+00 3.70273749e+00 2.70957649e+01
5.13025808e+00 4.42497474e+00 6.28651584e+00 1.21834305e+02
9.16529252e+00 1.01587926e+01 6.28651584e+00 7.14438451e+00
1.43430216e+00 1.43430216e+00 1.32410164e+01 1.17694156e+01
3.70273749e+00 2.96354630e+00 6.44249197e-01 8.40236584e+00
6.44249197e-01 4.78473706e+01 4.63452527e+01 6.44249197e-01
5.13025808e+00 1.01621834e+01 1.35271987e+02 1.43430216e+00
8.40236584e+00 0.00000000e+00 1.64758255e+01 2.68550191e+00
6.44249197e-01 7.41869904e+01 2.20740119e+00 1.62757692e-01
3.70273749e+00 5.81858748e+00 9.86718506e-01 9.67899648e+01
1.87171346e+01 0.00000000e+00 2.96354630e+00 3.63051376e+01
1.62757692e-01 5.13025808e+00 0.00000000e+00 9.16529252e+00
6.48996296e+00 2.68550191e+00 0.00000000e+00 3.09409154e+01
```

```
0.00000000e+00 9.86718506e-01 2.96354630e+00 6.28651584e+00
9.87837465e+01 2.96354630e+00 7.22915414e+00 1.62757692e-01]' has dtype
incompatible with int64, please explicitly cast to a compatible dtype first.
```

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[4.47834456e+00 6.73339750e+00 9.20115496e+00 1.10529355e+01
0.00000000e+00 3.22906641e+00 1.63282002e+01 2.78248316e+00
1.78498556e+01 2.26458133e+01 1.78498556e+01 5.26852743e+00
2.78248316e+00 3.22906641e+00 5.00481232e-02 9.29740135e+00
9.29740135e+00 3.51838306e+01 2.43214629e+01 4.36573628e+00
0.00000000e+00 1.19942252e+01 8.64196343e+00 1.06265640e+01
1.02040423e+01 5.54090472e+00 2.43214629e+01 7.40808470e+00
1.78498556e+01 5.54090472e+00 9.32627526e-01 0.00000000e+00
9.29740135e+00 2.26458133e+01 3.91126083e+01 0.00000000e+00
2.26458133e+01 2.13089509e+00 2.60356112e+01 2.60356112e+01
1.63282002e+01 2.26458133e+01 5.54090472e+00 5.86977863e+01
1.63282002e+01 5.54090472e+00 1.06265640e+01 1.94100096e+01
5.00481232e-02 2.10086622e+01 1.94100096e+01 5.26852743e+00
1.63282002e+01 1.87680462e+00 5.54090472e+00 2.78248316e+00
```

```

1.07122233e+00 4.47834456e+00 2.26458133e+01 8.04427334e+00
9.29740135e+00 1.06265640e+01 1.07122233e+00 9.29740135e+00
6.02021174e+00 6.73339750e+00 5.54090472e+00 9.20115496e+00
1.78498556e+01 0.00000000e+00 1.63282002e+01 6.75457170e+00
1.06265640e+01 1.14196343e+01 8.04427334e+00 1.20375361e+01
5.00481232e-02 4.36573628e+00 1.19942252e+01 8.00673725e+00
6.75457170e+00 0.00000000e+00 9.32627526e-01 1.19942252e+01
4.36573628e+00 6.75457170e+00 0.00000000e+00 1.87680462e+00
4.36573628e+00 1.07122233e+00 0.00000000e+00 1.63282002e+01
0.00000000e+00 9.29740135e+00 1.87680462e+00 1.48450433e+01
9.32627526e-01 1.63282002e+01 3.22906641e+00 9.29740135e+00]' has dtype
incompatible with int64, please explicitly cast to a compatible dtype first.

```

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[1.53770156e+01 1.51206096e+01 1.47398135e+01 3.35118379e+01
0.00000000e+00 1.29444471e+01 1.79625401e+00 2.03238152e+01
1.53301726e+01 1.04534156e+01 9.25446043e+00 1.11296334e+01
4.87656334e+01 4.18379110e+01 4.77517505e+01 1.36330111e+01
6.74377987e+00 1.53770156e+01 2.75332816e+00 1.51693086e+01

```

```

0.00000000e+00 3.29918442e+00 4.89000000e+02 8.16784376e+01
1.01657809e+01 5.08176075e+01 1.44574603e+02 7.91907145e+01
2.18593461e+01 3.62145708e+01 4.04136978e+00 0.00000000e+00
3.16753175e+00 3.29918442e+00 1.54340261e+01 0.00000000e+00
1.54040885e+01 1.20097656e+01 1.32302258e+00 3.52351443e-01
2.58393883e+01 1.54040885e+01 2.03876242e+02 8.04305413e+01
2.83242467e+01 1.30848147e+01 2.91686716e+01 1.15498406e+02
2.07655073e+02 1.76563861e+02 1.25271771e+01 1.45115294e+01
1.50217188e+01 4.98516973e+00 1.50451202e+01 2.58393883e+01
1.41085827e+01 1.17705475e+01 1.52309994e+01 1.30848147e+01
1.45088262e-01 4.04136978e+00 2.71850859e+00 1.54230920e+01
1.53418733e+01 1.50995484e+01 1.68582393e+00 5.69221275e+00
1.53770156e+01 0.00000000e+00 1.30848147e+01 3.99311866e+01
8.41721723e-01 3.28920496e+02 1.68582393e+00 3.44046795e+01
5.71671960e+01 4.46618466e+00 6.50597392e-01 9.19521067e+01
7.57420440e+00 0.00000000e+00 2.18593461e+01 2.72506137e+02
1.49615613e+01 1.47398135e+01 0.00000000e+00 1.39581283e+01
1.07329808e+01 5.71671960e+01 0.00000000e+00 1.52036440e+01
0.00000000e+00 4.88293012e+00 1.28859438e+01 8.41721723e-01
8.54705428e+01 1.41085827e+01 4.88293012e+00 2.50272410e+01]' has dtype
incompatible with int64, please explicitly cast to a compatible dtype first.

```

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame

```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!
You are setting values through chained assignment. Currently this works in
certain cases, but when using Copy-on-Write (which will become the default
behaviour in pandas 3.0) this will never work to update the original DataFrame
or Series, because the intermediate object on which we are setting values will
behave as a copy.
A typical example is when you are setting values in a column of a DataFrame,
like:
```

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: SettingWithCopyWarning:
A value is trying to be set on a copy of a slice from a DataFrame
```

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: Setting an item of
incompatible dtype is deprecated and will raise an error in a future version of
pandas. Value '[2.87247247e+01 3.43163163e+01 2.17757758e+01 1.99319319e+01
0.00000000e+00 3.58638639e+01 6.82722723e+01 1.93793794e+00
1.74614615e+01 1.53413413e+01 7.16716717e-01 8.23623624e+00
1.96216216e+01 1.74614615e+01 7.26126126e+00 2.98378378e+01
4.47207207e+01 4.47207207e+01 7.57497497e+01 3.46146146e+00
0.00000000e+00 6.10450450e+01 3.78000000e+02 6.24704705e+01
2.57277277e+01 2.07167167e+01 1.18666667e+02 9.22122122e+00
9.63623624e+01 3.46386386e+01 1.78178178e-01 0.00000000e+00
2.80280280e+00 6.68068068e+01 6.29629630e+00 0.00000000e+00
1.78178178e-01 2.29369369e+01 2.80280280e+00 7.72752753e+01
5.13473473e+01 1.53413413e+01 1.06306306e+00 1.87125125e+02
1.89261261e+02 6.16216216e+00 2.07167167e+01 2.34086086e+02
1.74614615e+01 3.22182182e+01 1.02162162e+01 2.62522523e+01
8.57657658e+00 1.52792793e+01 1.53413413e+01 2.40940941e+01
8.57657658e+00 5.33733734e+00 5.33733734e+00 2.05565566e+01
9.22122122e+00 8.98398398e+01 5.27027027e+01 3.65765766e+00
1.79979980e+01 2.77657658e+01 1.42662663e+02 1.01361361e+01
```

```

2.11711712e+01 0.00000000e+00 1.32612613e+01 1.93793794e+00
8.57657658e+00 6.10450450e+01 1.38598599e+01 1.16556557e+01
2.11711712e+01 1.52792793e+01 1.93793794e+00 1.48440440e+02
1.96216216e+01 0.00000000e+00 1.24004004e+01 2.17941942e+02
1.66586587e+01 1.86526527e+01 0.00000000e+00 1.96216216e+01
2.05565566e+01 1.93793794e+00 0.00000000e+00 5.13473473e+01
0.00000000e+00 3.65765766e+00 1.52792793e+01 1.02162162e+01
7.42342342e+01 6.16216216e+00 2.53653654e+00 8.57657658e+00]' has dtype
incompatible with int64, please explicitly cast to a compatible dtype first.

```

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```

X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError:
behaviour will change in pandas 3.0!

```

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
```

/tmp/ipykernel_4252/166556683.py:54: FutureWarning: ChainedAssignmentError: behaviour will change in pandas 3.0!

You are setting values through chained assignment. Currently this works in certain cases, but when using Copy-on-Write (which will become the default behaviour in pandas 3.0) this will never work to update the original DataFrame or Series, because the intermediate object on which we are setting values will behave as a copy.

A typical example is when you are setting values in a column of a DataFrame, like:

```
df["col"][row_indexer] = value
```

Use `df.loc[row_indexer, "col"] = values` instead, to perform the assignment in a single step and ensure this keeps updating the original `df`.

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

```
X_adjusted[col][mask] = X_adjusted[col][mask] * (Z_scores[mask,
X.columns.get_loc(col)] / max_Z[X.columns.get_loc(col)])
```

```
[20]: acc_holder_tame_outlier
```

```
[20]: {'IQR': 9.352301587301588,
      'LOF': 8.886904761904763,
      'iForest': 7.304347643097643,
      'SP': 9.75358108108108,
      'iNNe': 7.690771889400921}
```